

TECHNICAL HIRING ASSIGNMENT

Candidates are required to choose one of the two assignments below

This assignment is intended to evaluate practical engineering ability, code quality, performance awareness, and the ability to build a reliable user experience. Candidates should select the track that best represents the role they wish to be considered for.

Assignment Selection

Option	Role	Primary Focus
A	React Native Mobile Developer	Video-first mobile experience, smooth feed, Shorts, shared-element transitions and gestures
B	Web Developer	Real-time messaging, media support, moderation, reliability and security

Pro tip: We currently have lower representation in React Native. Choosing the React Native track may increase your probability of being considered for hiring.

General Instructions

- ✓ Choose ONE assignment only. Do not submit both unless requested separately.
- ✓ Use TypeScript wherever practical and keep the codebase structured, readable and maintainable.
- ✓ Open-source libraries may be used, but you should be able to explain important technical choices.
- ✓ Mock data is acceptable where a backend is not explicitly required. Real-time behavior in the Web assignment must be functional.
- ✓ Performance, reliability and interaction quality are more important than decorative UI.
- ✓ The implementation should be your own work. Documentation and technical references may be used as needed.

Submission Requirements

- ✓ GitHub repository containing the complete source code.
- ✓ README with setup instructions, architecture notes and key technical decisions.
- ✓ A working Android build / Expo preview for the mobile track, or a deployed web preview for the web track.
- ✓ A short screen recording demonstrating the required flows and features.

OPTION A - React Native Mobile Developer

Priority Assignment

Build a polished, video-first React Native experience with interaction quality comparable to a mature consumer video application. The main evaluation areas are smoothness, gestures, animation quality, video lifecycle management and practical mobile architecture.

1. Home Screen

Create the main video feed. It should remain smooth during fast scrolling and repeated navigation, rather than only looking correct in a static demo.

- ▶ Header with placeholder branding, search, notifications and profile/avatar.
- ▶ Horizontally scrollable category row with a clear selected state.
- ▶ Vertically scrolling video feed with at least 15-20 mock videos.
- ▶ Each video card should include thumbnail, duration, avatar, title, creator/channel name, view count, uploaded time and an options action.
- ▶ Use an efficient list implementation and avoid unnecessary re-renders during scroll.
- ▶ Images should be cached or otherwise handled efficiently enough that the feed does not visibly stutter.

2. Shorts Section

Add a Shorts preview section within the Home screen and a separate full-screen Shorts viewer.

- ▶ Horizontal Shorts preview cards on the Home screen.
- ▶ Tap a Short to open a full-screen vertical viewer.
- ▶ Vertical swipe between Shorts with smooth paging.
- ▶ Only the currently visible Short should actively play; off-screen videos should pause.
- ▶ Preload only what is useful for a smooth next/previous swipe instead of keeping every video active.
- ▶ Rapid repeated swiping should not cause overlapping audio, frozen players or obvious playback leaks.

3. Video Watch Screen

- ▶ Tapping a regular feed video should open a watch/detail screen with a functioning video player.
- ▶ Include a minimal title/creator/details area and enough surrounding content to demonstrate scrolling and gesture coordination.
- ▶ The player should preserve sensible playback state when the user interacts with the screen.

React Native Assignment - Interaction & Performance

4. Shared-Element-Style Transition

The transition from the Home feed to the watch screen is a core part of this assignment.

- ▶ The selected thumbnail/video container should visually expand or transform into the watch player instead of appearing through an abrupt screen change.
- ▶ Returning to the Home screen should feel visually connected to the originating item where practical.
- ▶ The implementation does not have to use a specific shared-element library; the quality and continuity of the interaction are what matter.

5. Gesture-Driven Player / Mini Player

- ▶ Dragging the watch screen downward should move the player with the user's finger in real time.
- ▶ The gesture must be interactive: finger movement should directly drive the UI rather than triggering a delayed animation after release.
- ▶ At a suitable threshold, the watch screen should minimize into a compact player near the bottom of the screen, or dismiss cleanly if you choose that interaction.
- ▶ Tapping the mini player should restore the full watch screen.
- ▶ Gesture conflicts with scrolling and other interactions should be handled intentionally.

6. Performance Expectations

- ▶ Use FlatList, FlashList or an equivalent virtualized approach appropriately.
- ▶ Use React Native Reanimated / Gesture Handler or equivalent tooling for high-frequency interactive animation where appropriate.
- ▶ Manage video mounting, playback and cleanup carefully so inactive videos do not continue consuming unnecessary resources.
- ▶ Use stable keys, memoization and sensible component boundaries to limit unnecessary rendering.
- ▶ The Home screen should still feel smooth after opening videos, returning, scrolling quickly and entering/exiting Shorts repeatedly.

What We Will Review

Area	What matters
Smoothness	Fast scrolling, rapid Shorts swipes, repeated open/close flows and absence of visible jank
Gestures & Animation	Shared-element-style continuity, finger-driven interactions and mini-player behavior
Video Lifecycle	Correct autoplay/pause behavior, cleanup, preloading strategy and no overlapping audio
Architecture	Readable TypeScript, sensible component structure and maintainable state management
UI Polish	Spacing, touch feedback, loading states and overall consistency

OPTION B - Web Developer

Real-Time Messaging System

Build a production-oriented messaging module for the web. The system should demonstrate real-time communication, media handling, moderation, database design, reliability and server-side security.

1. Real-Time Messaging

- ▶ One-to-one conversations are mandatory. Group conversations may be added as a bonus.
- ▶ Messages should appear for both users in real time without refreshing the page.
- ▶ Implement timestamps, delivery/read state, typing indicators, online/offline presence and unread counts.
- ▶ Load recent messages first and fetch older messages as the user scrolls upward.
- ▶ The system should be usable from two separate accounts simultaneously during the demo.

2. GIFs and Stickers

- ▶ Provide a GIF picker with search and preview using a suitable API or service.
- ▶ Provide a sticker picker with at least one usable sticker pack.
- ▶ GIFs and stickers should render inside the conversation without making the message list noticeably slow.

3. Image Upload & Nudity Detection

Images sent in chat must pass through a moderation step before they become visible to the recipient.

- ▶ Use a lightweight nudity / explicit-content detection model or service suitable for practical deployment.
- ▶ Unsafe images should be rejected before delivery and the sender should receive a clear moderation message.
- ▶ Document the model/service used, where inference runs, approximate model size if applicable, expected latency and the threshold/decision rule used.
- ▶ Do not implement moderation only in the browser. A user should not be able to bypass the check by directly calling the message API.

4. Profanity / Curse Word Moderation

- ▶ Detect and block prohibited profanity/curse words before the message is delivered.
- ▶ Enforcement must occur on the server side.
- ▶ Handle straightforward bypass attempts such as casing changes, added spaces, repeated letters and common character substitutions where reasonably practical.
- ▶ The goal is a clear, defensible moderation pipeline rather than a hardcoded client-side word check.

Web Assignment - Reliability, Performance & Security

5. Messaging Reliability

- ▶ Handle WebSocket disconnects and reconnection without silently losing already accepted messages.
- ▶ Prevent duplicate messages when a client retries or reconnects.
- ▶ Optimistic sending, temporary message IDs and idempotency are encouraged.
- ▶ The application should behave sensibly when the same account is open in more than one browser tab.

6. Performance & Data Handling

- ▶ Assume a conversation may contain 10,000+ messages. Do not load the entire history at once.
- ▶ Use cursor-based pagination or an equivalent efficient approach.
- ▶ Add appropriate database indexes for message retrieval and conversation membership queries.
- ▶ Media should be lazily rendered/loaded where practical.
- ▶ Large files should preferably be uploaded to object storage rather than passed through the application server as oversized request bodies.

7. Security Requirements

- ▶ Users must not be able to read conversations they do not belong to.
- ▶ Users must not be able to send a message as another account.
- ▶ Validate uploaded file type and size, and do not trust the filename alone.
- ▶ Apply authentication and authorization on the server side.
- ▶ Use reasonable rate limiting for message sends, uploads and moderation-sensitive endpoints.

Suggested Technology

You may choose your preferred implementation. A typical stack could include Next.js / React / TypeScript, PostgreSQL, Prisma, WebSockets or Socket.IO, Redis where needed, and S3 / Cloudflare R2 / equivalent object storage. The chosen architecture should be explained in the README.

What We Will Review

Area	What matters
Real-Time Behavior	Low-latency delivery, typing/read states and sensible reconnection behavior
Backend & Database	Message model, pagination, indexing, authorization and clear APIs
Moderation	Lightweight image moderation, server-side profanity blocking and resistance to simple bypasses
Reliability	Duplicate prevention, retries, idempotency and multi-session behavior
UI / UX	Usable responsive chat layout, media picker behavior and clear message states

Final note: the task is intended to show engineering judgment. A smaller implementation that is stable, well-structured and demonstrably smooth/reliable will be valued more than a larger feature set that only works in ideal conditions.